# Project Extensions Proposal - From Attitude + Flatness to a Complete Autonomy Stack

*Ten graded extensions building on this repo's validated base (two papers implemented, four simulators, 70 tests, independent paper validation), each with the maths, the exact implementation plan in our codebase, the validation story, and the effort estimate. Group 12 - quadrotor-project.*

## 0. What we already have (the foundation everything builds on)

- Paper 1 (Fresk 2013) - the nonlinear quaternion attitude law reproduced bit-exactly, deployed unmodified to MuJoCo, gym-pybullet-drones, Gazebo and real ArduPilot firmware, cross-engine agreement 0.03 deg RMS.
- Paper 2 (Choutri 2017) - differential flatness + double-loop LQR implemented equation-by-equation, validated by independent review (17/17 findings confirmed, bugs fixed), tracking the paper's circle to 2.7-2.8 cm radial RMS on two physics plants plus a noise-robustness run (3.7 cm).
- Infrastructure most student projects never reach: 70-test suite, animated in-engine recordings, interactive sims with live HUDs, a browser ground station, deployment pages - and a validation culture (every claim measured).

> **What "big volume of work" should mean here**
>
> Volume that impresses a reviewer is not ten half-features - it is a coherent story where each extension makes the previous ones more valuable, every result is measured against a baseline we already own, and each addition produces code + tests + plots + a report section. The plan below is ordered so that even the first three extensions alone read as a complete project arc: more trajectories, better tracking, quantified robustness.

## 1. The recommended roadmap

| Phase | Extensions | Why in this order | Effort |
|---|---|---|---|
| A - Breadth (weeks 1-3) | E1 trajectory library + min-snap; E2 aggressive paths (figure-8, yaw-follow) | Instant visible volume on the existing 3-plant benchmark; E1's optimizer feeds everything after | S + M |
| B - Depth (weeks 3-6) | E3 full flatness feedforward (omega_d, tau_ff); E4 third controller family (geometric SE(3)) benchmark | Closes a documented gap (measurable win) and turns the repo into a controller comparison study - the analytically strongest story | M + M |
| C - Autonomy (weeks 6-9) | E6 global planning (A*/RRT* through the city) into the tracker; E5 wind/disturbance suite | The flagship: perception-free full autonomy loop (plan -> smooth -> track) with honest robustness numbers | L + S |

| Phase | Extensions | Why in this order | Effort |
|---|---|---|---|
| D - Stretch (weeks 9+) | E7 iLQR on flat outputs; E8 energy-aware profiles; E9 ArduPilot HIL; E10 mission-planner UI | Pick by interest; any one of these distinguishes the project at review | M/L each |

## 2. E1 - Trajectory library with minimum-snap optimization

Why it impresses. Our tracker currently flies one circle. A trajectory LIBRARY (hover, point-to-point, circle, figure-8, polynomials through waypoint lists) plus a real optimizer is what distinguishes a control reproduction from an autonomy stack - and minimum-snap is the classic quadrotor result (Mellinger & Kumar, ICRA 2011) that maps 1:1 onto our flatness stack.

The maths. Flatness says the input depends on derivatives of the flat outputs - physically, the third derivative of position (jerk) drives body rates and the fourth (snap) drives torques. So the smoothest flyable path minimizes snap:

```
min  ∫ ‖d⁴p/dt⁴‖² dt     subject to:  waypoint positions (and optional
                                       velocities/accelerations) at knot times
 solution: piecewise polynomials, solved as ONE quadratic program (QP)
in the polynomial coefficients - closed form for unconstrained segments
```

*Min-snap: flatness turns "smooth input" into "small 4th derivative of the output" - an optimization purely in the flat space.*

Plan in our repo. New flatness/trajectories additions: a PolyTrajectory class (numpy polynomial pieces with exact analytic derivatives up to snap - our FlatRef already carries p, v, a; extend with jerk, snap); a min_snap module solving the QP via scipy (no new dependencies: solve as unconstrained linear system with continuity constraints, or cvxopt-free KKT); waypoint scenarios. Validation: unit tests on continuity (C3 at knots), a benchmark comparing min-jerk vs min-snap tracking error at fixed speed on MuJoCo, plots of rotor effort vs smoothness. Effort: M (2 weeks).

## 3. E2 - Aggressive trajectories now: figure-8, yaw-follow, vertical orbit

Why it impresses. Cheapest visible win: three new sigma(t) functions, each exercising a different flatness stress (yaw coupling, sign reversal, thrust-direction swings), all on the existing 3-plant benchmark with the existing recorder producing in-engine GIFs.

The maths. Each is a flat-output choice; the map does the rest:

```
              figure-8:   x = A·sin(ωt),  y = B·sin(2ωt)
        yaw-follow:  ψ = atan2(ẏ, ẋ)    heading along the tangent
          orbit:  horizontal circle with  z(t) = z0 + r·sin(ωt)
          (thrust vector sweeps a cone - tilt-construction stress test)
```

*New FlatRef generators only - zero controller changes. That IS the flatness pitch demonstrated by our own code.*

Plan: three generators + tests (start-at-rest ramps as we already do) + benchmark table extension + recordings. Effort: S (3-4 days).

# 4. E3 - Complete the flatness feedforward (omega_d and tau_ff)

Why it impresses. Our independent validation flagged, correctly, that we implement Eq. (15)'s thrust-vector and orientation rows but regulate body rate to zero instead of feeding forward the parameterized omega_d. Completing the chain is a principled upgrade with a measurable prediction: tracking error on curved paths should drop, because the inner loop stops fighting its own reference dynamics.

The maths. Differentiate the reference attitude:

$$\dot{q}_d = \tfrac{1}{2}\, q_d \otimes [0,\ \omega_d] \quad \Rightarrow \quad \omega_d = 2\cdot \mathrm{vec}\left( q_d^{-1} \otimes \dot{q}_d \right)$$

$$\tau_{ff} = J\cdot\dot{\omega}_d + \omega_d \times (J\cdot\omega_d) \qquad \text{flatness input row, Eq. 15}$$

$$\text{inner loop:} \quad \tau = \tau_{ff} - K\cdot[q_{vec,err},\ \omega - \omega_d]$$

*omega_d needs sigma up to the 3rd derivative (jerk) - which E1's polynomial library supplies analytically. E3 and E1 reinforce each other.*

Plan: extend FlatRef with jerk; implement omega_d/tau_ff in flatness.py; A/B benchmark (same paths, feedforward on/off) on the ideal model and MuJoCo; report the delta. Effort: M (1-2 weeks).

# 5. E4 - Third controller family: geometric SE(3) tracking, benchmarked

Why it impresses. "We reproduced two papers" becomes "we implemented a third controller family from the research literature and benchmarked all three on identical paths, plants and noise" - that is a genuine comparative study, the kind of table that anchors a report chapter.

The maths. Lee, Leok & McClamroch (2010) track position errors $b_x = R^T(x - x_d)$, velocity errors, and attitude errors as bimaps on SO(3), giving an almost-global exponential stability proof:

$$m\cdot\ddot{x} = -k_x\cdot e_x - k_v\cdot e_v + m\cdot g\cdot\hat{z} + \|f_d\|\cdot b3_d \qquad \text{(position law)}$$

$$e_R = \tfrac{1}{2}\,(R_d^T R - R^T R_d)^\vee, \quad \tau = -k_R\cdot e_R - k_\Omega\cdot e_\Omega + \text{feedforward}$$

$$\text{guarantee: almost-global exponential tracking (no linearization)}$$

*The geometric law needs no hover assumption - the axis the LQR papers linearize around is exactly where it is strongest.*

Plan: new flatness/geometric.py implementing the SE(3) law against the same flatness references (the map is controller-agnostic); benchmark P2 (Fresk) vs double-loop LQR (Choutri) vs geometric on circle + figure-8 + orbit, three plants, with/without noise; one comparison table + discussion of where each wins (this is also a ready-made report chapter and viva defence). Effort: M (2 weeks).

# 6. E5 - Wind and disturbance robustness suite

Why it impresses. Every result so far is in still air. A quantified robustness story - the same Dryden-style wind the repo's interactive sim already has (WIND_LEVELS), now measured against the path tracker - is what a reviewer asks for next, and we can answer with numbers.

The maths/plan. Apply wind as a world-frame force disturbance f_w(t) = base + gust amplitudes (already implemented in the MuJoCo app); run the flatness tracker across the wind ladder at fixed circle speed; report tracking RMS vs wind level; optionally add a disturbance observer (first-order ESO on the acceleration error) and show the RMS drop. Effort: S without observer, M with (1-2 weeks).

# 7. E6 - The flagship: global path planning through the city

Why it impresses. This is the extension that makes the project title literal: a drone that PLANS around obstacles, converts the plan to a flyable trajectory, and tracks it - all in the existing MuJoCo city scene, with the existing recorder producing the video. Planning -> smoothing -> tracking is the full autonomy loop.

The maths/plan. Three layers, each small on its own:

- Grid/graph over the city scene (occupancy from the scene XML's known obstacles); A* (admissible, optimal) and RRT* (asymptotically optimal) implementations from scratch - a few hundred lines each, no dependencies.
- Corridor extraction and resampling of the geometric path to waypoints with velocity limits (safe interval from max tilt => max speed per segment).
- E1's min-snap through the waypoints -> flatness -> existing tracker. Validate: planning time, path length vs optimal, tracking error, and a full-city mission video (start pad -> around buildings -> land).

```
plan  --A*/RRT*-->  geometric path  --resample-->  waypoints

--min-snap QP-->  sigma(t)  --flatness-->  (T_d, q_d, omega_d, tau_f
                                            f)

  --LQR loops-->  rotor speeds  --MuJoCo city-->  measured flight
```

*The full pipeline. Each arrow is a module we will own end to end.*

Effort: L (3 weeks). Highest wow-per-line of the proposal.

# 8. E7 to E10 - stretch extensions (pick by interest)

## E7 - iLQR / receding-horizon MPC on the flat model

Implement iterative LQR from scratch on the flat 4-state-per-axis model with input limits (thrust and tilt as box constraints on derivatives), run it as 200 Hz MPC on the circle and figure-8, and compare against the LQR baseline - optimality under constraints is the exact axis where LQR is weakest. From-scratch iLQR (no solver dependency) is a strong algorithms talking point. Effort: M/L.

## E8 - Energy-aware trajectory profiles

Use the motor model (kf, km, ESC lag - all already in our model) to compute power per segment, integrate battery discharge along candidate speed profiles for the same path, and pick the energy-optimal one; report the speed-vs-energy Pareto curve. Connects the project to endurance, the practical question. Effort: M.

## E9 - ArduPilot SITL hardware-in-the-loop for the flatness stack

The repo already compiles and flies real ArduPilot firmware under WSL2 with the attitude law. Extending the bridge so the flatness/LQR stack commands position setpoints to real firmware (GUIDED missions) turns the path planner into a HIL demonstration - the strongest systems-engineering item. Effort: L.

### E10 - Interactive mission planner in the browser GCS

The pysitl ground station already exists (websocket, modes, live telemetry). Add click-to-place waypoints on a 2D city map, live flatness-trajectory preview, mission upload, and a live tracking view. Pure demo value; makes every other extension presentable in one interface. Effort: M.

## 9. How to present this work (report + demo + viva)

- Report skeleton - Part I attitude reproduction (exists), Part II flatness path planning (exists), Part III comparative controllers (E4), Part IV autonomy pipeline (E1+E6), Part V robustness (E5). Each part ends with its measured table - never a claim without a number, matching the repo's validation culture.
- Demo script - one MuJoCo window, five minutes: paper tests pinned (X/Z/B), circle tracking, figure-8 with yaw-follow, city mission planned around buildings, then the same mission under HIGH wind. Every segment has an in-engine recording as backup.
- Viva anchors - the control-rate stability bound (derived, not tuned); the knife-edge flip analysis; the desaturation divergence (with the runaway plot); the ESC-lag stability finding; the body-vs-world rate bug our validation caught. Each is a story where WE found and fixed something real.
- Course mapping - optimal control (LQR, Riccati, iLQR), nonlinear systems (flatness, SE(3)), planning (A*, RRT*, smoothing), robotics systems (multi-rate loops, mixers, failsafes), software engineering (70 tests, multi-sim CI-style validation).

## 10. Scope discipline - what NOT to do

- No new frameworks - everything above is numpy/scipy + what we already have. A new ROS/Gazebo-heavy stack would consume weeks producing screenshots, not results.
- No perception (cameras/VIO) in this cycle - it would dominate the timeline and push control/planning content out. It is the natural next project.
- No gain re-tuning per extension - the frozen-controller discipline is the repo's identity; comparisons must hold controllers fixed and vary scenarios.
- Ship each extension as: module + tests + benchmark row + plot/recording + README section. An extension without its measurement does not count as done.

All effort labels assume the existing repo (flatness package, MuJoCo/gym adapters, recorder, test suite). Every extension's validation reuses the three-plant benchmark so results stay comparable across the whole project.